

```
#!/usr/bin/env bash
# =====
# SNS STACK BOOTSTRAP - Cold-start the full system in one command
#
# Usage:
#   chmod +x sns_boot.sh
#   ./sns_boot.sh           # full stack
#   ./sns_boot.sh --no-ui   # API + harness only (headless / server)
#   ./sns_boot.sh --dry     # validate env without starting processes
#
# Stops cleanly on Ctrl+C - kills all child processes.
# =====

set -euo pipefail

# --- Colour palette -----
GOLD='\033[38;5;172m'
GRN='\033[0;32m'
RED='\033[0;31m'
DIM='\033[2m'
RST='\033[0m'

# --- Flags -----
NO_UI=false
DRY_RUN=false
for arg in "$@"; do
    case $arg in
        --no-ui) NO_UI=true ;;
        --dry) DRY_RUN=true ;;
    esac
done

# --- Banner -----
echo ""
echo -e "${GOLD} ════════════════════════════════════════════════════════════════ ${RST}"
echo -e "${GOLD} || STUDIO NERVOUS SYSTEM // STACK BOOTSTRAP v1.0 || ${RST}"
echo -e "${GOLD} || n+3 Capital Architecture - Full System Cold Start || ${RST}"
echo -e "${GOLD} ════════════════════════════════════════════════════════════════ ${RST}"
echo ""

# --- PID tracking (for clean shutdown) -----
PIDS=()

cleanup() {
```

```

echo ""
echo -e "${GOLD}>> SHUTDOWN SIGNAL RECEIVED - terminating all processes...${RST}
for pid in "${PIDS[@]}"; do
    kill "$pid" 2>/dev/null && echo -e "${DIM}    killed PID $pid${RST}"
done
echo -e "${GRN}>> SNS STACK OFFLINE - clean exit.${RST}"
exit 0
}
trap cleanup SIGINT SIGTERM

# — Env validation —————
check_env() {
    echo -e "${GOLD}[1/5] VALIDATING ENVIRONMENT...${RST}"

    # .env file
    if [ ! -f .env ]; then
        if [ -f .env.example ]; then
            echo -e "${DIM}    .env not found - copying from .env.example${RST}"
            cp .env.example .env
        else
            echo -e "${RED}[FAIL] No .env or .env.example found.${RST}"
            echo -e "${DIM}    Create .env with: SNS_INTERNAL_SECRET, PORT, ALLOWED_
            exit 1
        fi
    fi

    # Load env vars
    set -o allexport
    # shellcheck disable=SC1091
    source .env
    set +o allexport

    # Required vars
    if [ -z "${SNS_INTERNAL_SECRET:-}" ] || [ "${SNS_INTERNAL_SECRET}" = "your_high
        echo -e "${RED}[WARN] SNS_INTERNAL_SECRET is unset or still default.${RST}"
        echo -e "${DIM}    System will use 'dev_secret' fallback. Change before pr
    fi

    # Node
    if ! command -v node &>/dev/null; then
        echo -e "${RED}[FAIL] Node.js not found. Install from https://nodejs.org${RST}
    fi
    NODE_VER=$(node -v)
    echo -e "${GRN}    Node:    ${NODE_VER}${RST}"

    # Python
    PYTHON_CMD=""

```

```

for cmd in python3 python; do
    if command -v "$cmd" &>/dev/null; then PYTHON_CMD="$cmd"; break; fi
done
if [ -z "$PYTHON_CMD" ]; then
    echo -e "${RED}[FAIL] Python not found.${RST}"; exit 1
fi
PY_VER=$( "$PYTHON_CMD" --version 2>&1 )
echo -e "${GRN}          Python:  ${PY_VER}${RST}"

echo -e "${GRN}          ENV OK${RST}"
}

# — Node dependencies —————
install_node_deps() {
    echo ""
    echo -e "${GOLD}[2/5] NODE DEPENDENCIES...${RST}"
    REQUIRED="express cors helmet express-rate-limit dotenv"
    MISSING=""
    for pkg in $REQUIRED; do
        if [ ! -d "node_modules/$pkg" ]; then MISSING="$MISSING $pkg"; fi
    done
    if [ -n "$MISSING" ]; then
        echo -e "${DIM}          Installing:${MISSING}${RST}"
        npm install $MISSING --save 2>&1 | tail -3
    else
        echo -e "${GRN}          All Node deps present — skipping install.${RST}"
    fi
}

# — Python dependencies —————
install_python_deps() {
    echo ""
    echo -e "${GOLD}[3/5] PYTHON DEPENDENCIES...${RST}"
    "$PYTHON_CMD" -c "import requests, rich, dotenv" 2>/dev/null || {
        echo -e "${DIM}          Installing: requests rich python-dotenv${RST}"
        "$PYTHON_CMD" -m pip install requests rich python-dotenv --quiet
    }
    echo -e "${GRN}          Python deps ready.${RST}"
}

# — Launch API server —————
start_api() {
    echo ""
    echo -e "${GOLD}[4/5] LAUNCHING SNS API CORE...${RST}"
    PORT="${PORT:-3001}"

    # Check if port is already bound

```

```

if lsof -i ":$PORT" &>/dev/null 2>&1; then
    echo -e "${DIM}          Port $PORT already in use — assuming server running, ski
    return
fi

node server.js > logs/server.log 2>&1 &
API_PID=$!
PIDS+=("${API_PID}")

# Wait up to 6s for health endpoint
echo -ne "${DIM}          Waiting for API"
for i in $(seq 1 12); do
    sleep 0.5
    if curl -sf "http://localhost:$PORT/health" &>/dev/null; then
        echo -e "${RST}"
        echo -e "${GRN}          API ONLINE → http://localhost:$PORT   [PID $API_PID]${R
        return
    fi
    echo -ne "."
done

echo -e "${RST}"
echo -e "${RED}[WARN] API did not respond in 6s. Check logs/server.log${RST}"
}

# — Launch harness —————
start_harness() {
    echo ""
    echo -e "${GOLD}[5/5] LAUNCHING SIMULATION HARNESS...${RST}"
    sleep 1 # Brief pause so API is fully warm before first injection

    "$PYTHON_CMD" sns_simulation_harness.py > logs/harness.log 2>&1 &
    HARNESS_PID=$!
    PIDS+=("${HARNESS_PID}")
    echo -e "${GRN}          Harness running [PID $HARNESS_PID] — tailing log below.${R
}

# — Launch React dev server (optional) —————
start_ui() {
    if $NO_UI; then return; fi
    echo ""
    echo -e "${GOLD}[UI]   LAUNCHING REACT DEV SERVER...${RST}"

    # Detect package manager and start command
    if [ -f "package.json" ]; then
        if grep -q '"dev"' package.json; then
            npm run dev > logs/ui.log 2>&1 &

```

```

    else
        npm start > logs/ui.log 2>&1 &
    fi
    UI_PID=$!
    PIDS+=("$UI_PID")
    echo -e "${GRN}      React dev server [PID $UI_PID] - check logs/ui.log${RST}"
else
    echo -e "${DIM}      No package.json found - skipping React boot.${RST}"
    echo -e "${DIM}      Mount SNSWarRoomDashboard.jsx in your existing React app"
fi
}

# — Status summary —————
print_status() {
    echo ""
    echo -e "${GOLD}===== ${RST}"
    echo -e "${GOLD}  SNS STACK ACTIVE                               ${RST}"
    echo -e "${GOLD}===== ${RST}"
    echo -e "  API Core:    ${GRN}http://localhost:${PORT:-3001}/api/sns/vault-stat"
    echo -e "  Harness:    ${GRN}logs/harness.log${RST}"
    if ! $NO_UI; then
        echo -e "  UI:         ${GRN}http://localhost:5173  (or :3000)${RST}"
    fi
    echo -e "  Logs:       ${DIM}logs/server.log  logs/harness.log${RST}"
    echo ""
    echo -e "${DIM}  Press Ctrl+C to shut down all processes cleanly.${RST}"
    echo -e "${GOLD}===== ${RST}"
    echo ""
}

# — Main —————
mkdir -p logs

check_env

if $DRY_RUN; then
    echo ""
    echo -e "${GRN}>> DRY RUN COMPLETE - environment validated. No processes starte"
    exit 0
fi

install_node_deps
install_python_deps
start_api
start_harness
start_ui
print_status

```

```
# — Tail harness output to stdout while stack runs —————  
tail -f logs/harness.log &  
PIDS+=($!)  
wait
```